IMEN574 / IDSC574 · PROGRAMMING FOR AX

22

AI for Data Science

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 Programming with AI

Lectures 2–4

02 Python and Classical Machine Learning

Lectures 5–13

03 **Deep Learning and Foundation Models**

Lectures 14–22

Outline: AI for Data Science

01 **Tabular Learning**

Match model priors to tables

02 **Time-Series Forecasting**

Preserve the forecast origin

03 **Automating Experiments**

Search without moving the test

01 · Tabular Learning

01 **Tabular Learning**
Match model priors to tables

02 **Time-Series Forecasting**
Preserve the forecast origin

03 **Automating Experiments**
Search without moving the test

Why Data Modality Matters

A foundation model inherits assumptions from its training tasks

Data	Structure	Strong baseline
Table	Heterogeneous columns	Linear model or trees
Time series	Ordered history	Seasonal naive
Text or image	Pretrained representation	Frozen encoder

Tabular Prediction Contract

Columns, rows and target timing define the task

Schema	age	balance	default
Row A	23	100	0
Row B	42	80	1

A column is usable only if it exists at prediction time

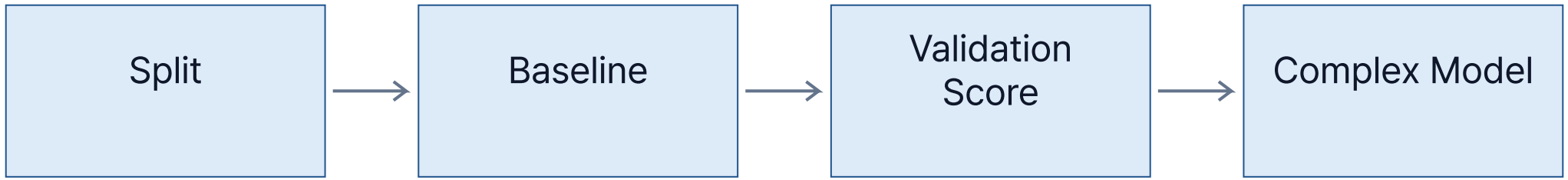
Heterogeneous Features

Numeric scale and category identity need different treatment

Column	Type	Operation
Temperature	Numeric	Scale if needed
Machine ID	Categorical	Encode identity
Timestamp	Temporal	Respect ordering

Tree Baselines

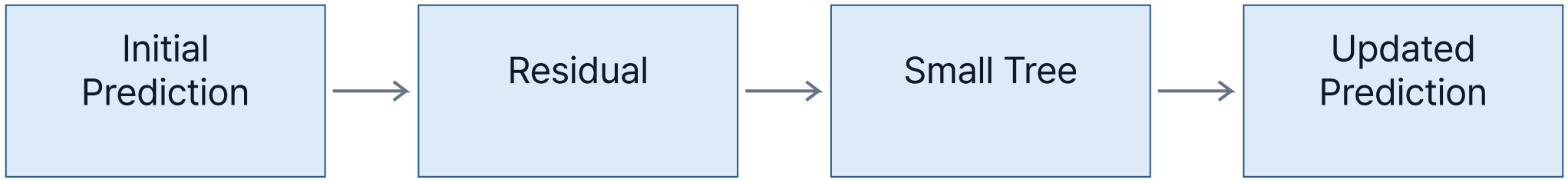
A simple model establishes the value of added complexity



A baseline anchors the comparison

Gradient Boosting

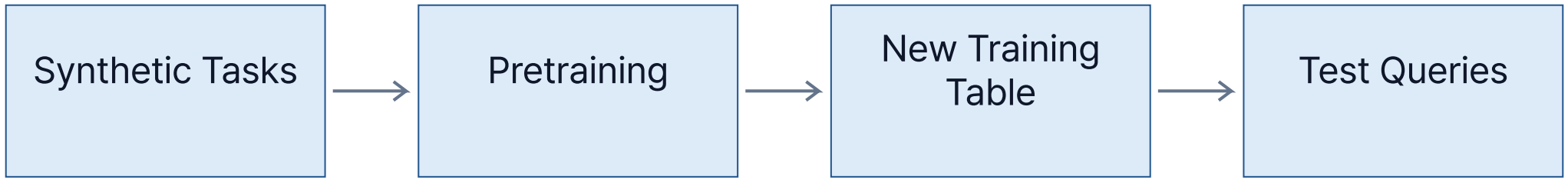
Successive trees correct remaining prediction error



A boosted model accumulates targeted corrections

Tabular Foundation Models

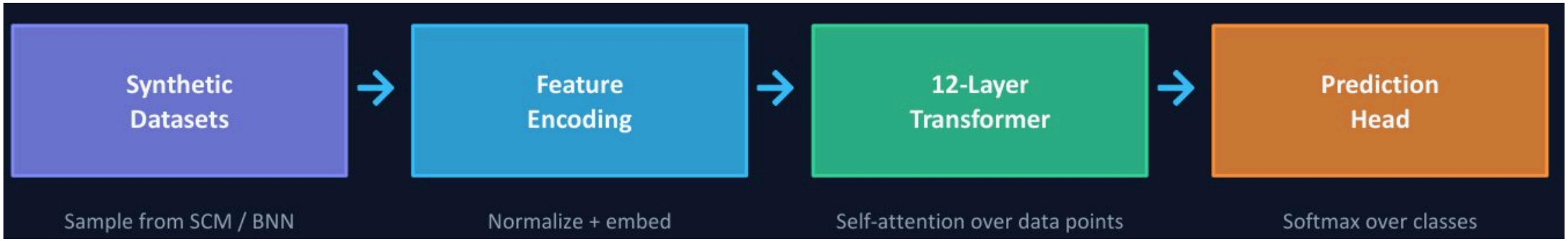
Pretraining can supply a prior over supervised table tasks



https://arxiv.org/abs/2207.01848

Prior-Fitted Networks

Task simulation moves part of the learning effort into pretraining



The pretraining task distribution supplies the prior

https://arxiv.org/abs/2207.01848

Inference-Time Context

Training rows and query rows play different roles

Train row 1	x1	y1
Train row 2	x2	y2
Query row	x*	?

The query target must remain hidden

Caveat: Prior Mismatch

A learned task prior may not cover the deployment regime

Change	Check
More rows or columns	Supported limits
New target type	Model scope
Missing patterns	Validation slice
New domain	Held-out data

Demo: Trees and TabPFN

Measured scores on fixed held-out splits

Code | Results

Run in Colab ↗

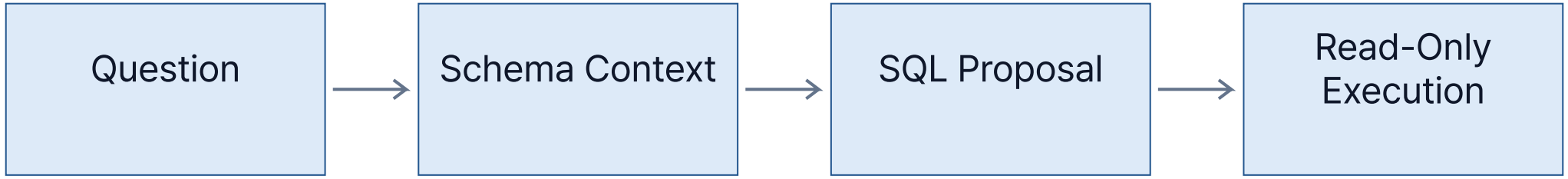
```
model = TabPFNClassifier(device="cuda" if torch.cuda.is_available() else "cpu",
n_estimators=1, random_state=42)
model.fit(X_tr, y_tr)
prediction = model.predict(X_te)
print("Held-out accuracy:", accuracy_score(y_te, prediction))
```

Compare scores, data size, and total runtime. Is one method always best?

TabPFN v2 uses one estimator; the bounded CPU benchmark is not a model ranking

Text-to-SQL and Table QA

A fluent query must still match the data schema and intent



The database verifies syntax while task checks verify intent

Demo: Execute the Proposed SQL

Run the proposed query on an actual in-memory SQLite table

Inspect the top two Engineering salaries; change the query to count each department

SQLite query

```
SELECT name, salary FROM employees
WHERE dept = 'Engineering'
ORDER BY salary DESC
LIMIT 2;
```

Run SQL | Reset query

```
name | salary
Bob | 95000
Alice | 80000
```

SQLite executes the query; no language model generates SQL here

Answer: Rows and Aggregates

Check the executed rows and the grouping unit

```
Engineering top two: Bob 95000, Alice 80000

Engineering: n = 2, mean_salary = 87500
Sales:       n = 2, mean_salary = 65000
```

A Table Model Decision

Compare predictive value, resource cost and auditability

Criterion	Evidence
Prediction	Same validation split
Cost	Measured runtime and memory
Scope	Supported task and input
Audit	Repeatable preprocessing

Worked Example: Fit and Hold Out

Trace the inputs before execution

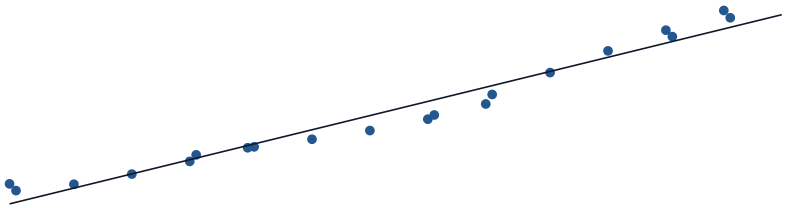
```
Fixed 120 rows: x=((37i mod120)/60)−1
y=2x+.7x²+.15sin(2.3i)
Fit rows0–19; hold out rows80–119; compare mean and least squares.
```

Demo: Fit and Hold Out

Executable educational calculation; no pretrained inference

Increase training rows from 4 to 80; compare least squares with the mean baseline

Training rows 20 Fit model Least squares



Fixed synthetic data; held-out rows 80–119

Train MAE = 0.189

Held-out MAE = 0.193

Mean baseline MAE = 1.021

Fitted slope=1.960, intercept=0.245; held-out n=40.

Questions: Does more data always improve this sample? Has a foundation model actually run?

Answer: Fit and Hold Out

Same inputs as the initial demo state

```
Fitted slope1.959574, intercept .245351
train MAE .188605
held-out MAE .192513
held-out mean-baseline MAE1.020612
```

Failure: Fit and Hold Out

Identify the assumption that breaks

```
This is an actual least-squares fit, not TabPFN or a tree benchmark
repeated held-out inspection turns that set into validation
```

02 · Time-Series Forecasting

01 Tabular Learning

Match model priors to tables

02 **Time-Series Forecasting**

Preserve the forecast origin

03 Automating Experiments

Search without moving the test

Why Time Order Matters

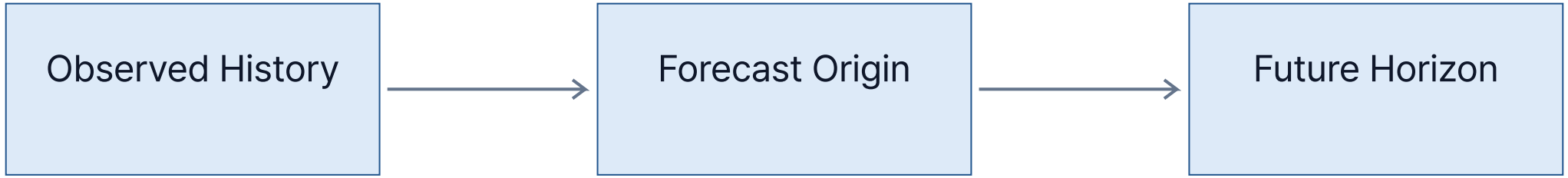
Future observations cannot help a past forecast

Time	t1	t2	t3	t4	t5
Split	train	train	train	test	test

The evaluation origin separates history from the future

Forecast Horizon

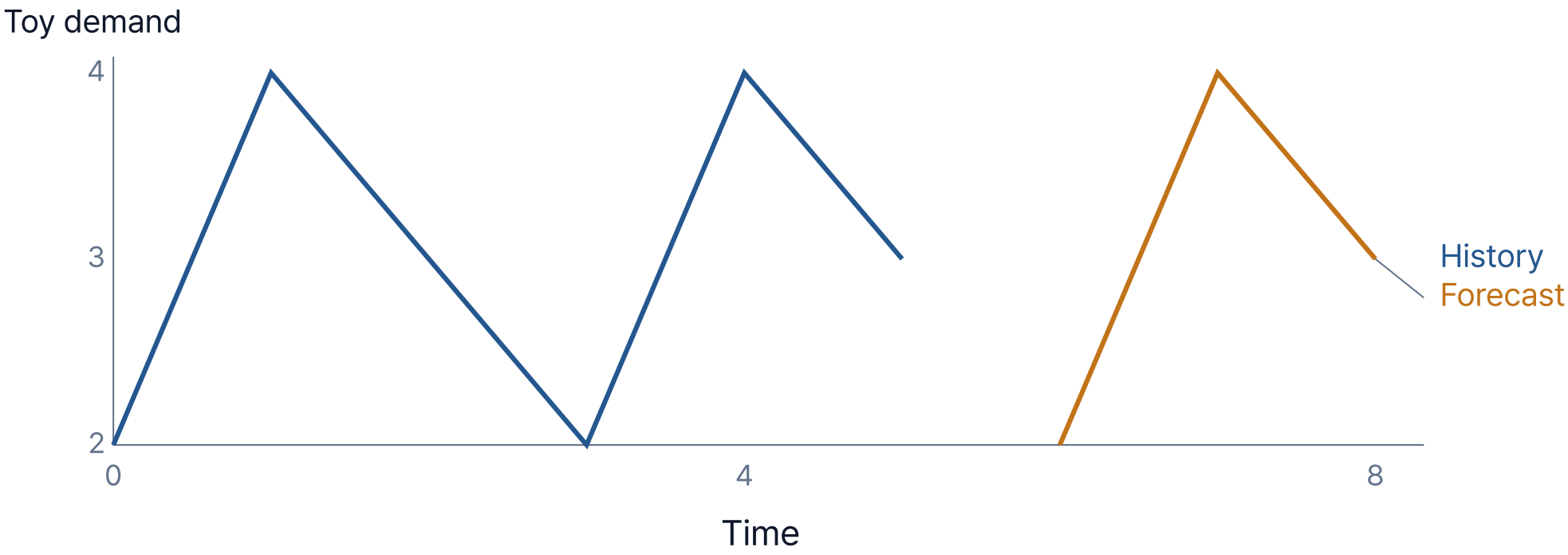
The target is a future path with a stated length



A forecast is tied to an origin and horizon

Seasonal Naive Baseline

Repeat the last observed seasonal pattern before fitting complexity



The last cycle supplies a transparent forecast

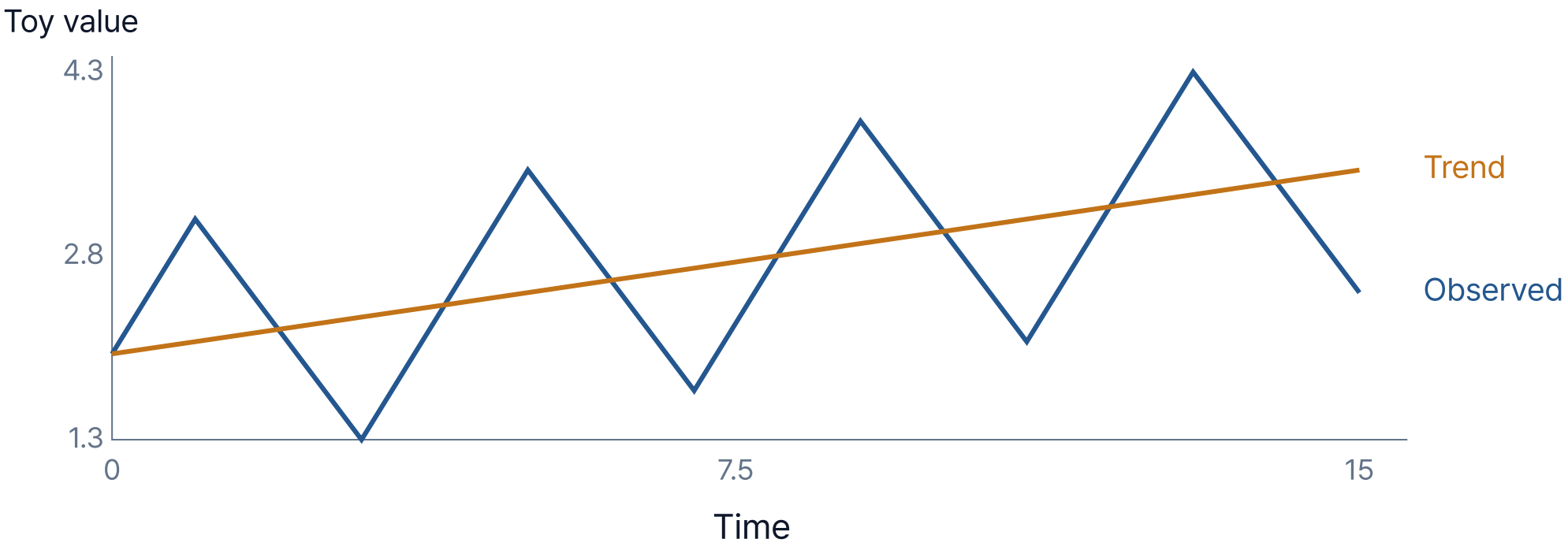
Decomposition View

Trend, seasonality and residuals describe different variation

$$y_t = g(t) + s(t) + h(t) + \varepsilon_t$$

Decomposition Components

A structured forecast separates recurring and nonrecurring effects



The observed series combines a rising trend and a repeating seasonal component

Rolling-Origin Evaluation

Move the forecast origin while preserving temporal order

	t1	t2	t3	t4	t5
Origin 1	T	T	T	V	-
Origin 2	T	T	T	T	V

Each evaluation uses only the history available at its origin

Worked Example: Backtest and Leakage

Trace the inputs before execution

```
y[t]=[2,4,3,1][t mod4]+.05t
Origins20,24,28,32; horizon4; seasonal baseline ŷ[t]=y[t−4]
At each origin, use only already observed values.
```

Demo: Backtest and Leakage

Executable educational calculation; no pretrained inference

Add a shift at t=32; compare a valid seasonal forecast with leaked future targets

Feature access Past season only Shift at t=32 0

Origins 20, 24, 28, 32; horizon 4; season 4

Origin 20: 0.200 Origin 24: 0.200 Origin 28: 0.200 Origin 32: 0.200

Rolling-origin MAE = 0.200

VALID baseline: predict y[t] using y[t−4]

Series = [2,4,3,1] seasonal cycle + .05t + optional shift.
Questions: Why is perfect MAE invalid with future access? Which origin encounters the shift?

Answer: Backtest and Leakage

Same inputs as the initial demo state

```
Each origin MAE .2
mean rolling-origin MAE .2
```

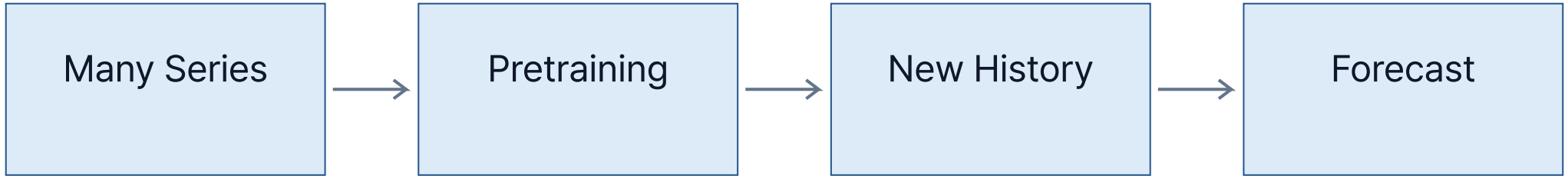
Failure: Backtest and Leakage

Identify the assumption that breaks

```
Future targets make a perfect score meaningless
a distribution shift may invalidate historical averages
```

Time-Series Foundation Models

Pretraining can transfer patterns across series



Transferred patterns condition a forecast for a new series

https://arxiv.org/abs/2310.10688

Patch-Based Forecasting

A sequence model can process time-series patches

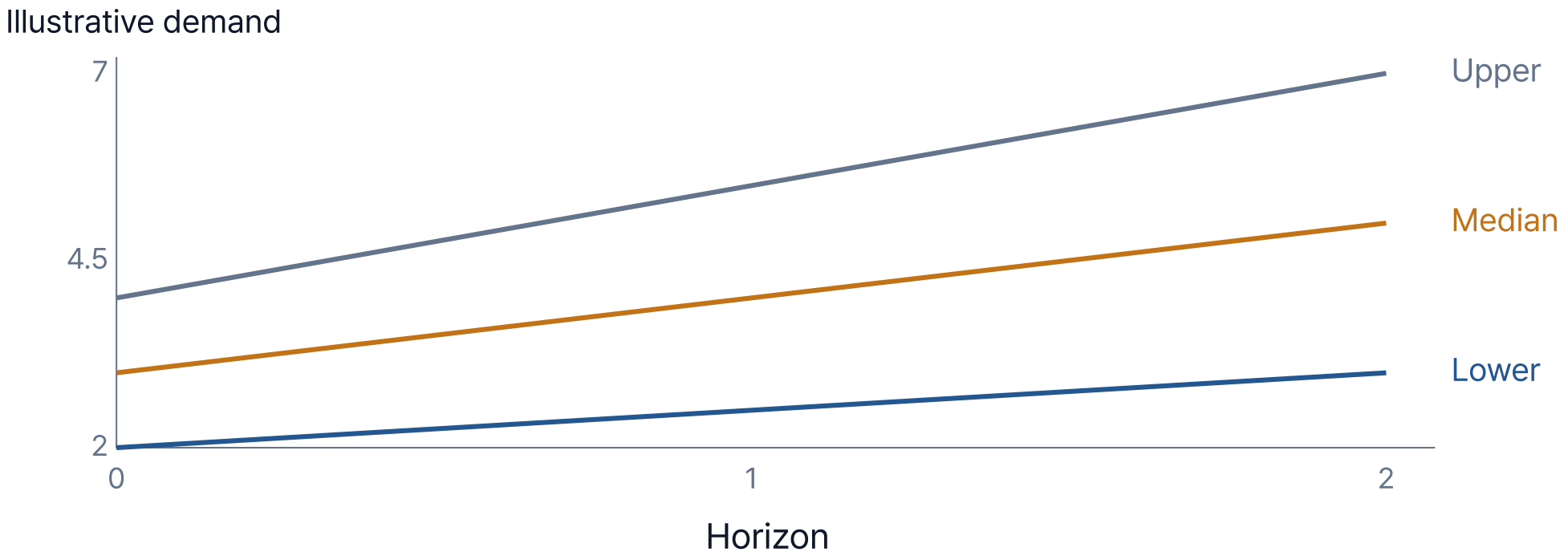


Time patches form the forecasting context

https://arxiv.org/abs/2310.10688

Quantiles and Uncertainty

A forecast distribution supports several decision thresholds



Forecast intervals widen in this illustrative example

Demo: Prophet and Chronos

Actual forecasts from the same observed context

Code | Results

Run in Colab ↗

```
samples = pipeline.predict(context, prediction_length=horizon, num_samples=100)
low, median, high = np.quantile(samples.detach().cpu().numpy()[0], [0.1, 0.5,
0.9], axis=0)
print("Forecast horizon:", len(median))
print("Held-out MAE:", np.abs(median - actual).mean())
```

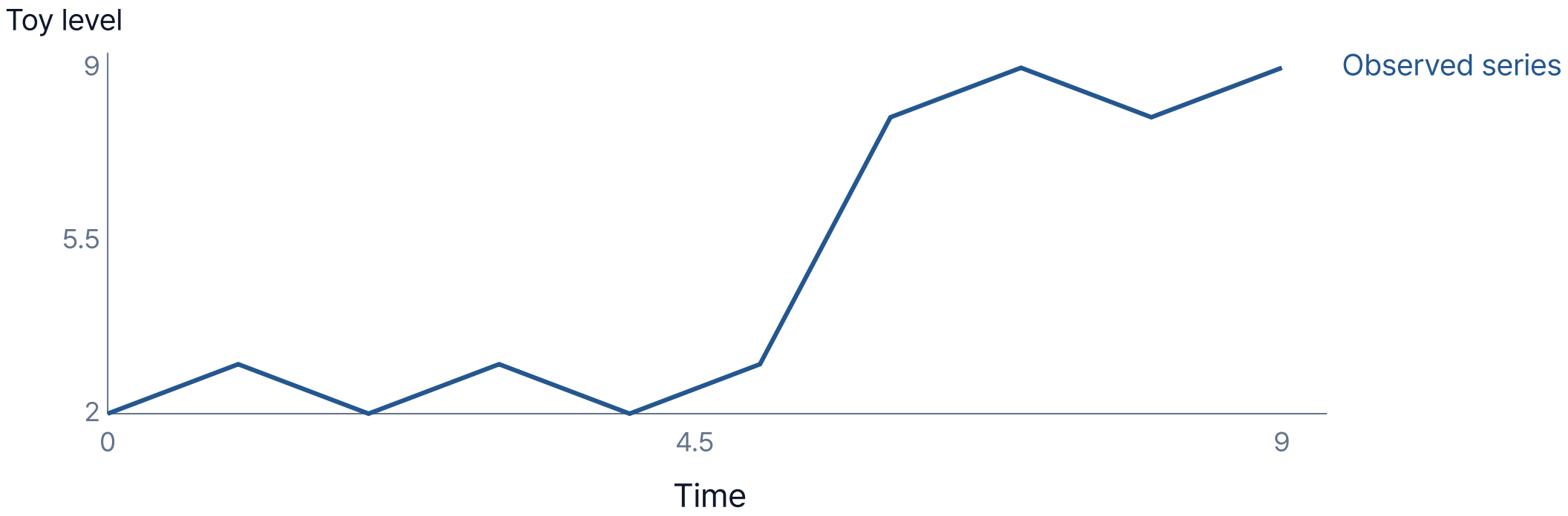
Compare medians, intervals, and held-out errors. Which observations were visible?

Synthetic sales data; the comparison shares the context window and forecast horizon

https://huggingface.co/amazon/chronos-t5-mini

Distribution Shift

A new regime can invalidate an otherwise useful history



The historical level no longer matches the new regime

Caveat: Future Covariates

A forecast feature must be known at the forecast origin

Feature	Available?
Calendar day	Yes
Scheduled price	If published
Future actual demand	No
Revised final sensor data	Often no

03 · Automating Experiments

01 Tabular Learning

Match model priors to tables

02 Time-Series Forecasting

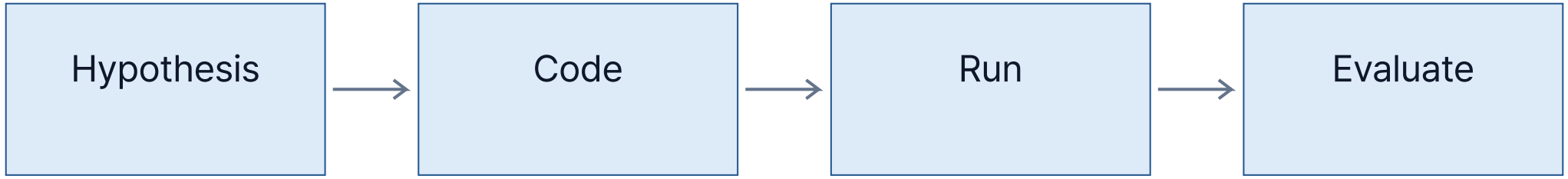
Preserve the forecast origin

03 **Automating Experiments**

Search without moving the test

Why an Experiment Agent

Automating model trials also automates the opportunity for leakage



Every automated trial must preserve the evaluation contract

Immutable Evaluation Contract

Freeze the split, metric and test boundary before search

Rule 1	Split	fixed
Rule 2	Metric	fixed
Rule 3	Test labels	sealed
Rule 4	Budget	bounded

Search changes candidates while the evaluation contract stays fixed

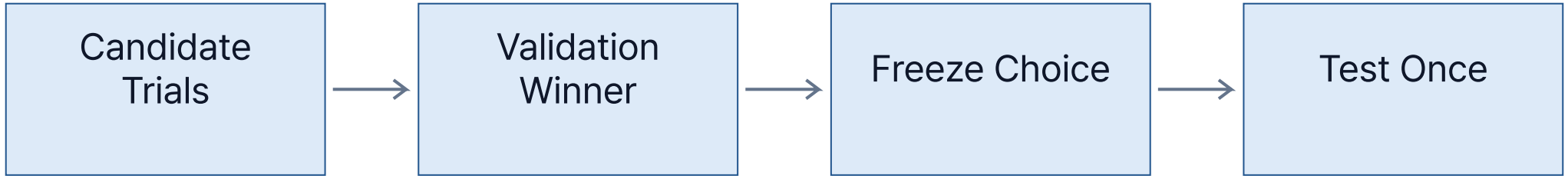
Candidate Registry

Each trial has a reproducible configuration and result

Trial	Model	Validation
A	Seasonal naive	0.80 MAE
B	Ridge	0.72 MAE
C	Small tree	0.76 MAE

Evidence Before Promotion

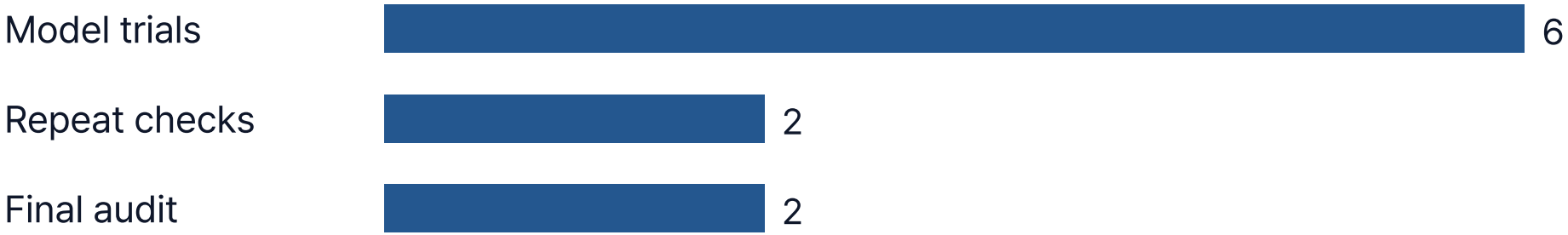
A validation winner needs one final held-out check



The test set evaluates a frozen selection procedure

Budget Allocation

Search breadth competes with repeatability and verification time

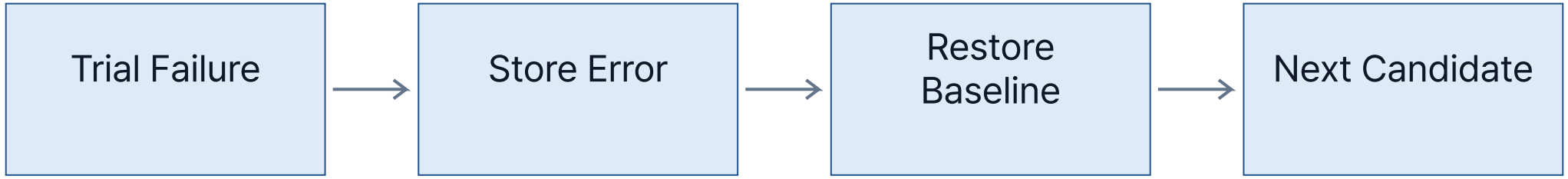


Illustrative ten-run budget

Some budget must remain for checking the chosen result

Failure Recovery

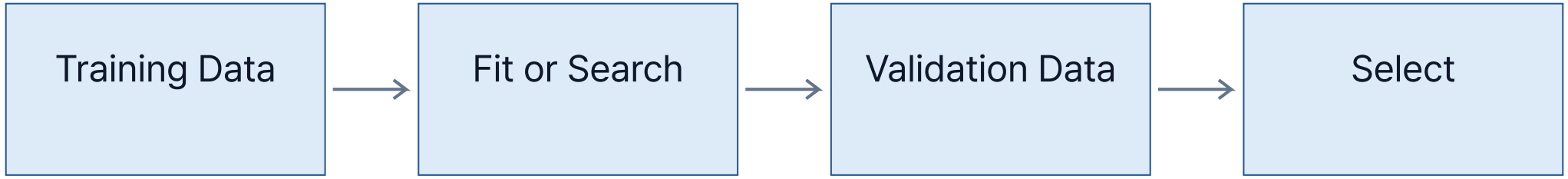
An exception should record the failed trial rather than erase it



A failed trial remains part of the experiment record

Data Access Boundaries

The experiment agent needs training access, not hidden test answers



Data permissions should enforce the intended split

Reproducibility Record

Keep code, data identity, seed and environment with the score

Artifact	Purpose
Code revision	Recreate the method
Data fingerprint	Recreate the input
Seed and environment	Interpret variation
Metric implementation	Recreate the score

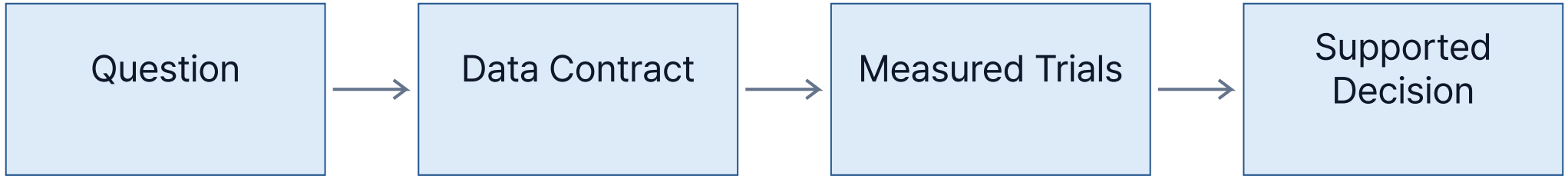
Human Decision Point

A better score does not automatically justify deployment

Question	Decision evidence
Error cost	Task loss
Shift	New-domain evaluation
Runtime	Measured constraints
Explanation	Inspection and limits

A Verified AI Workflow

The output is a repeatable experiment with a defensible conclusion



A useful workflow connects a question to reproducible evidence

Takeaway: AI for Data Science

01 A strong baseline makes added complexity measurable

02 Forecasts use only information available at the origin

03 An experiment agent must preserve the evaluation contract